



☕ Master Guide: JUnit 5 Testing Framework

5 JUNIT5

JAVA

TESTING

FRAMEWORK



© 2026 Avinash Dhanuka

Master Guide: Unit Testing with JUnit 5

Crafted with ♥ for Test-Driven Development



GITHUB

AVINASH-706



CONTACT ME VIA GMAIL

Learning Note: This guide provides a complete understanding of JUnit 5 testing framework, from basic assertions to advanced parameterized testing, with real-world examples and internal execution flow.

Table of Contents

1. [What is JUnit? \(The Foundation\)](#)
 2. [Why Do We Need JUnit?](#)
 3. [Project Structure Deep Dive](#)
 4. [JUnit Lifecycle & Annotations](#)
 5. [Assertion Methods \(The Core\)](#)
 6. [Parameterized Testing \(Advanced\)](#)
 7. [Internal Execution Flow](#)
 8. [Topics Covered in This Project](#)
 9. [Spring Boot vs Plain Java Structure](#)
 10. [Interview Questions](#)
-

1. WHAT IS JUNIT? (THE FOUNDATION)

Definition

JUnit is an open-source **Unit Testing Framework** for Java. It is used to write and run repeatable automated tests to ensure that individual units of code (methods, classes) work as expected.

Key Characteristics

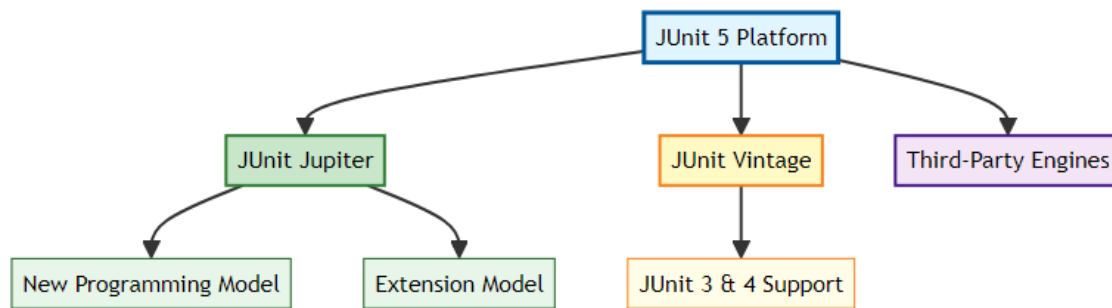
- **Framework Type:** Testing Framework (Not a library)
- **Current Version:** JUnit 5 (Jupiter) - Released in 2017
- **Architecture:** Modular (JUnit Platform + JUnit Jupiter + JUnit Vintage)
- **Annotations-Based:** Uses Java annotations like `@Test`, `@BeforeEach`
- **Assertion Library:** Built-in assertion methods for validation

JUnit Evolution

Version	Year	Key Features
JUnit 3	2000	Test classes extend <code>TestCase</code> , method names start with <code>test</code>
JUnit 4	2006	Annotation-based (<code>@Test</code>), no need

Version	Year	Key Features
JUnit 5	2017	to extend classes Modular architecture, Lambda support, Parameterized tests

🏠 JUnit 5 Architecture



Components Explained: 1. **JUnit Platform:** Foundation for launching testing frameworks on the JVM 2. **JUnit Jupiter:** New programming model and extension model for JUnit 5 3. **JUnit Vintage:** Provides backward compatibility with JUnit 3 and 4

2. WHY DO WE NEED JUNIT?

🔗 The Problem Without Testing

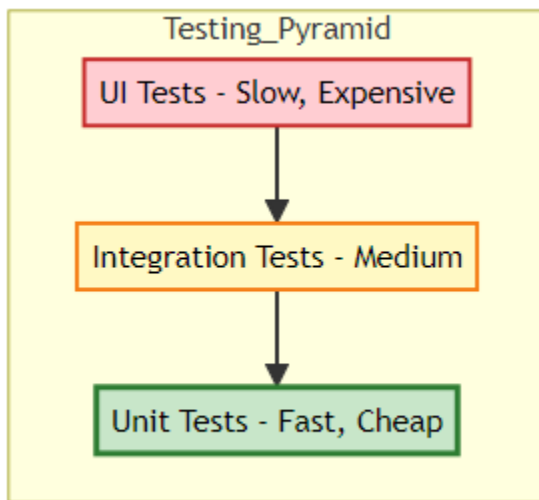
Imagine you have a Calculator class with 10 methods. Every time you make a change: - ✗ You manually test each method by running `main()` - ✗ You write `System.out.println()` statements everywhere - ✗ You forget to test edge cases (divide by zero, negative numbers) - ✗ When bugs appear in production, you don't know which method broke

✓ The Solution: Automated Testing with JUnit

1. **Automated Verification:** Write tests once, run them thousands of times
2. **Regression Prevention:** Ensure new code doesn't break existing functionality

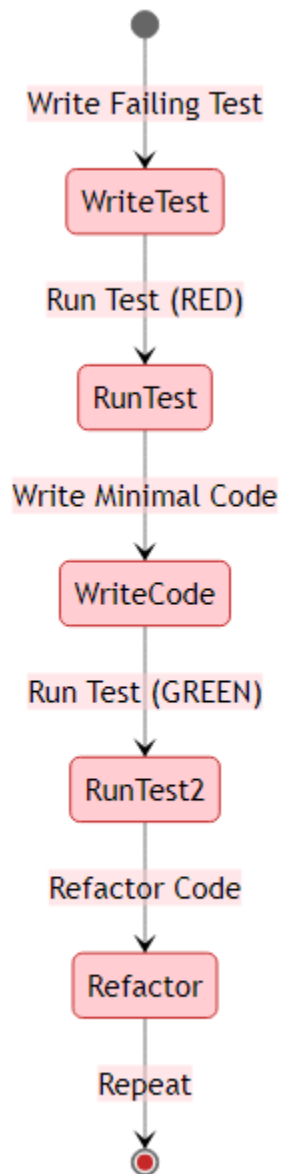
3. **Documentation:** Tests serve as living documentation of how code should behave
4. **Confidence:** Refactor code fearlessly knowing tests will catch errors
5. **CI/CD Integration:** Automated tests run in build pipelines before deployment

📈 Testing Pyramid



JUnit focuses on the base: Unit Tests (70% of all tests should be unit tests)

🔄 Test-Driven Development (TDD) Cycle



3. PROJECT STRUCTURE DEEP DIVE

📄 **Documentation by:** Avinash Dhanuka | [GitHub Profile](#)

📁 Complete Folder Hierarchy

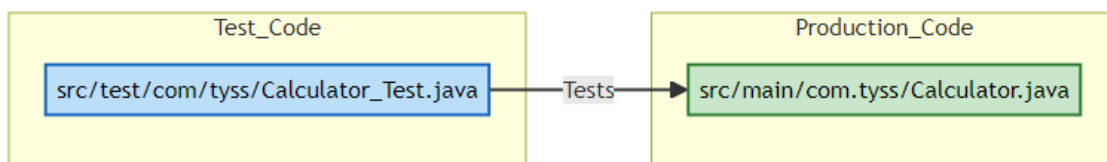
```
JUnitOne/
├── .idea/                # IntelliJ IDEA configuration files
├── out/                  # Compiled .class files (bytecode)
├── src/
│   ├── main/             # Production code
│   │   ├── com.tyss/     # Package structure
│   │   │   ├── Calculator.java
│   │   │   └── StudentEligibilityService.java
│   │   └── test/         # Test code
│   │       ├── com/tyss/ # Mirror package structure
│   │       │   ├── Calculator_Test.java
│   │       │   └── StudentServiceTest.java
│   │       └── resources/ # Test resources
│   │           └── calculator-data.csv
├── .gitignore
└── JUnitOne.iml          # IntelliJ module file
```

📁 Package Structure Explained

Why com.tyss?

Java follows **reverse domain naming convention**: - If your company domain is tyss.com - Package becomes com.tyss - This ensures global uniqueness (no package name conflicts)

Main vs Test Separation



Critical Rule: Test classes must be in the **same package** as the class they test (but in the test folder)

📁 How Packages Work in Java

Package Declaration (Line 1 of every file):

```
package com.tyss; // This file belongs to com.tyss package
```

What Happens Internally: 1. **Compilation:** javac creates folder structure matching package name 2. **Bytecode Location:** Calculator.class is placed in out/com/tyss/ 3. **Import Resolution:** JVM searches for classes using package path

Example:

```
// File: Calculator.java
package com.tyss; // Declares package

public class Calculator {
    // Class code
}
```

When compiled:

```
out/
├── com/
│   └── tyss/
│       └── Calculator.class ← Bytecode stored here
```

🔍 File-by-File Breakdown

1. Calculator.java (Production Code)

Location: src/main/com.tyss/Calculator.java

Purpose: Business logic for arithmetic operations

Key Methods:

```
public static int add(int a, int b)        // Addition
public static int divide(int a, int b)     // Division with exception handling
public static boolean isEven(int a)        // Even number check
```

Why Static? No need to create object instances for simple utility methods.

2. StudentEligibilityService.java (Production Code)

Location: src/main/com.tyss/StudentEligibilityService.java

Purpose: Business logic for student eligibility validation

Key Method:

```
public boolean isEligible(int age) // Returns true if age >= 18
```

Exception Handling: Throws IllegalArgumentException for negative age

3. Calculator_Test.java (Test Code)

Location: src/test/com/tyss/Calculator_Test.java

Purpose: Comprehensive testing of Calculator class

Topics Covered: - ✓ Lifecycle annotations (@BeforeAll, @AfterEach) - ✓ Basic assertions (assertEquals, assertTrue) - ✓ Exception testing (assertThrows) - ✓ Parameterized tests (9 different sources!) - ✓ Test watchers (custom test result tracking) - ✓ Repeated tests (@RepeatedTest) - ✓ Disabled tests (@Disabled)

Reference: See [Calculator_Test.java:21](#) for lifecycle implementation

4. StudentServiceTest.java (Test Code)

Location: src/test/com/tyss/StudentServiceTest.java

Purpose: Testing StudentEligibilityService

Topics Covered: - ✓ All basic assertions - ✓ assertAll() for grouped assertions - ✓ fail() method for manual test failure - ✓ Object reference testing (assertSame, assertNotSame)

5. calculator-data.csv (Test Resource)

Location: src/test/resources/calculator-data.csv

Purpose: External test data for @CsvFileSource

Format:

```
2,3,5      ← input1, input2, expected_output  
10,5,15  
20,4,24
```

Why External Files? - Separates test data from test logic - Easy to modify without changing code - Can be managed by non-developers (QA team)

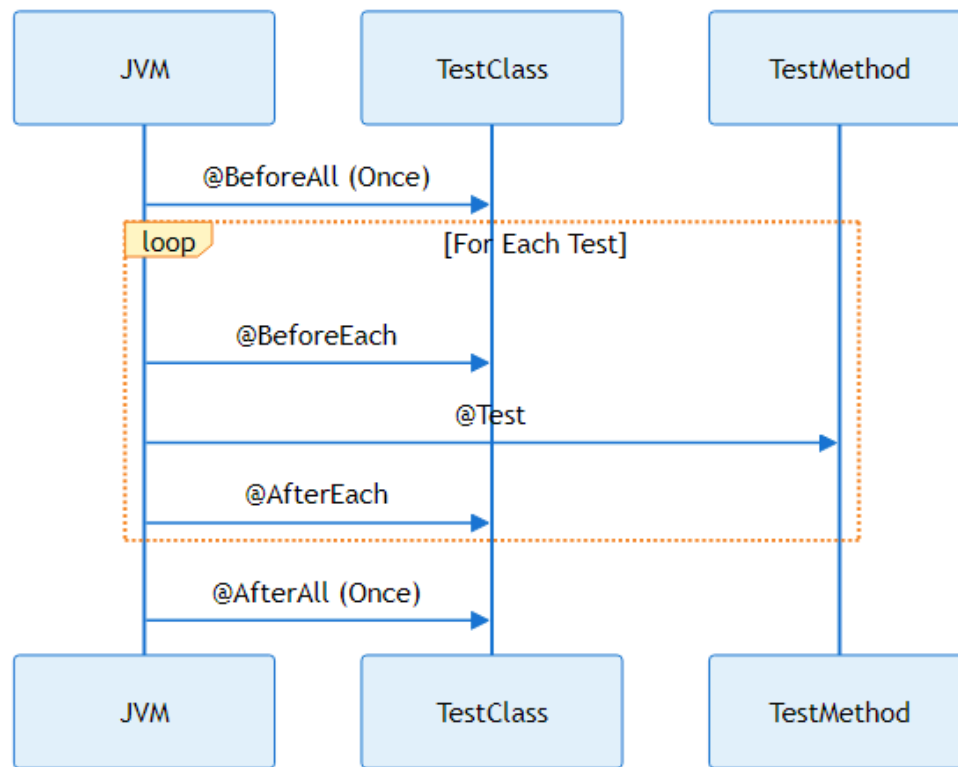
4. JUNIT LIFECYCLE & ANNOTATIONS

✦ What are Annotations?

Annotations are **metadata** that provide information to the compiler and JVM about how to execute code.

Syntax: @AnnotationName

🔄 Test Execution Lifecycle



📁 Core Lifecycle Annotations

Annotation	Execution	Method Type	Use Case
@BeforeAll	Once before all tests	static	Database connection, load config
@BeforeEach	Before each test	Instance	Create fresh object instances
@Test	Test execution	Instance	Actual test logic
@AfterEach	After each test	Instance	Cleanup resources, reset state
@AfterAll	Once after all	static	Close connec-

Annotation	Execution	Method Type	Use Case
	tests		tions, print summary

🔍 Real Example from Calculator_Test.java

Reference: [Calculator_Test.java:46](#)

```

@BeforeAll
static void beforeAllTests() {
    System.out.println("-- Start All Tests --");
}

@BeforeEach
void setup() {
    calc = new Calculator(); // Fresh instance for each test
}

@AfterEach
void tearDown() {
    calc = null; // Cleanup
}

```

Why Fresh Instances? Ensures test isolation - one test's changes don't affect another.

🔗 Test Annotations

Annotation	Purpose	Example
@Test	Marks method as test	@Test void testAdd() {}
@DisplayName	Custom test name	@DisplayName("Division Success Test")
@Disabled	Skip test execution	@Disabled @Test void skip() {}
@RepeatedTest(n)	Run test n times	@RepeatedTest(5)

Reference: [Calculator_Test.java:217](#)

```
@RepeatedTest(2)
void testRepeated() {
    assertEquals(4, calc.add(2, 2)); // Runs twice
}
```

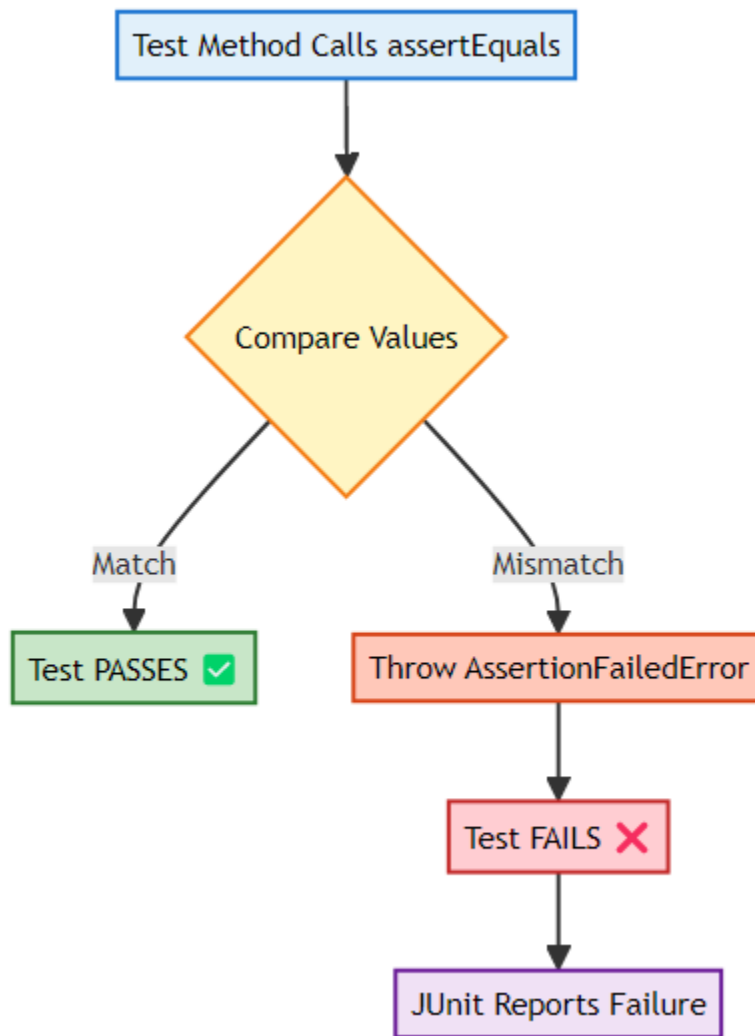
5. ASSERTION METHODS (THE CORE)

✦ What is an Assertion?

An **assertion** is a statement that checks if a condition is true. If false, the test fails.

Syntax: `assertXxx(expected, actual, message)`

Internal Working of Assertions

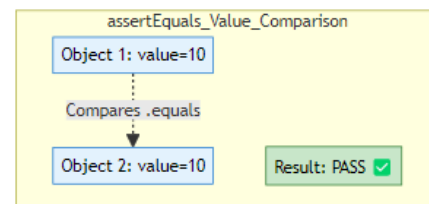
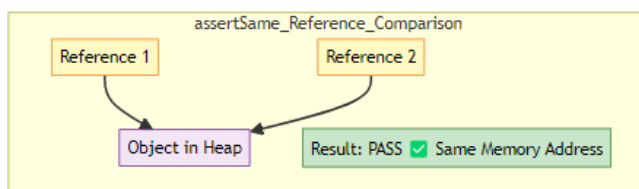


Complete Assertion Methods Table

Method	Purpose	Example	Reference
assertEquals	Check equality	<code>assertEquals(5, calc.add(2,3))</code>	Calculator Test.java:16
assertNotEquals	Check inequality	<code>assertNotEquals(10, calc.subtract(8,3))</code>	Calculator Test.java:16
assertTrue	Check boolean	<code>assertTrue(calc.isEven(4))</code>	Calculator Test.java:17

Method	Purpose	Example	Reference
assertFalse	Check boolean false	<code>assertFalse(calc.isEven(5))</code>	Calculator_Test.java:17
assertNull	Check if null	<code>assertNull(obj)</code>	Calculator_Test.java:18
assertNotNull	Check if not null	<code>assertNotNull(calc)</code>	Calculator_Test.java:18
assertSame	Check same reference	<code>assertSame(ref1, ref2)</code>	Calculator_Test.java:19
assertNotSame	Check different reference	<code>assertNotSame(new Calc(), new Calc())</code>	Calculator_Test.java:20
assertThrows	Check exception thrown	<code>assertThrows(ArithmeticException.class, ...)</code>	Calculator_Test.java:20
assertAll	Group multiple assertions	<code>assertAll(() -> ..., () -> ...)</code>	StudentServiceTest.java
fail	Manually fail test	<code>fail("Should not reach here")</code>	StudentServiceTest.java

🔍 Deep Dive: assertEquals vs assertEquals



Example:

```
Integer a = 100;
```

```
Integer b = 100;
assertEquals(a, b);    // ✔ PASS (values are equal)
assertSame(a, b);     // ✔ PASS (Integer cache: same object)

Integer x = 200;
Integer y = 200;
assertEquals(x, y);    // ✔ PASS (values are equal)
assertSame(x, y);     // ✘ FAIL (different objects in heap)
```

🔗 assertThrows: Exception Testing

Why Important? Ensures your code handles errors correctly.

Reference: Calculator Test.java:208

```
@Test
void testAssertThrows() {
    assertThrows(ArithmeticException.class, () -> {
        calc.divide(10, 0); // Should throw exception
    });
}
```

Internal Flow: 1. JUnit executes the lambda () -> calc.divide(10, 0) 2. If ArithmeticException is thrown → Test PASSES ✔ 3. If no exception or different exception → Test FAILS ✘

6. PARAMETERIZED TESTING (ADVANCED)

📖 **Comprehensive Guide by:** Avinash Dhanuka | © 2026

🚀 What is Parameterized Testing?

Running the **same test logic** with **different input data** without duplicating code.

🔗 The Problem Without Parameterization

```
// ✘ BAD: Repetitive code
@Test void testAdd1() { assertEquals(5, calc.add(2, 3)); }
```

```

@Test void testAdd2() { assertEquals(0, calc.add(0, 0)); }
@Test void testAdd3() { assertEquals(5, calc.add(-5, 10)); }
// ... 100 more tests? 🤖

```

✓ The Solution: @ParameterizedTest

```

// ✓ GOOD: One test, multiple data sets
@ParameterizedTest
@CsvSource({"2,3,5", "0,0,0", "-5,10,5"})
void testAdd(int a, int b, int expected) {
    assertEquals(expected, calc.add(a, b));
}

```

📋 All 9 Parameterized Sources Covered

Source	Use Case	Example	Reference
@CsvSource	Multiple parameters inline	@CsvSource({"2,3,5"})	Calculator Test
@ValueSource	Single parameter	@ValueSource(ints={2,4,6})	Calculator Test
@MethodSource	Complex/dynamic data	@MethodSource("provideData")	Calculator Test
@CsvFileSource	External CSV file	@CsvFileSource(resources="/data.csv")	Calculator Test
@NullSource	Null values	@NullSource	Calculator Test
@EmptySource	Empty strings/collections	@EmptySource	Calculator Test
@NullAndEmptySource	Both null and empty	@NullAndEmptySource	Calculator Test
@EnumSource	Enum values	@EnumSource(Operation.class)	Calculator Test

🔍 Deep Dive: Each Source Type

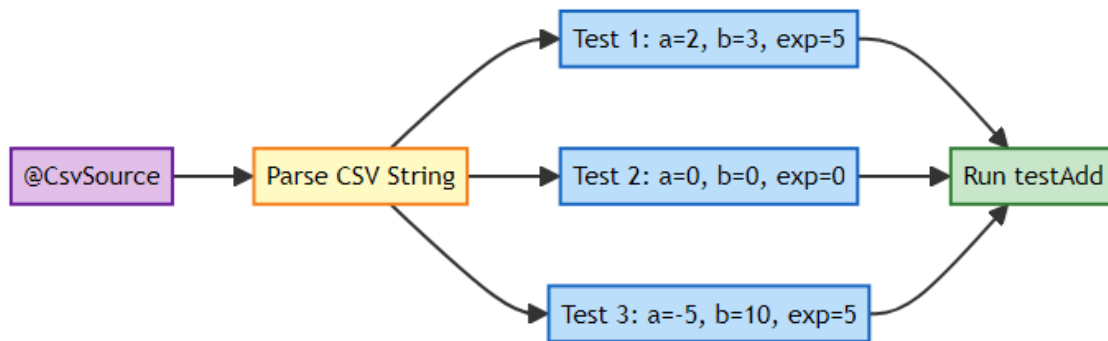
📄 @CsvSource (Comma-Separated Values)

When to Use: Multiple parameters with expected output

Reference: [Calculator_Test.java:64](#)

```
@ParameterizedTest
@CsvSource({
    "2, 3, 5",      // Test case 1
    "0, 0, 0",      // Test case 2
    "-5, 10, 5"     // Test case 3
})
void testAdd(int a, int b, int expected) {
    assertEquals(expected, calc.add(a, b));
}
```

Internal Execution:



📄 @ValueSource (Single Parameter)

When to Use: Testing with single input values

Reference: [Calculator_Test.java:77](#)

```
@ParameterizedTest
@ValueSource(ints = {2, 4, 6, 8, 10})
void testIsEven(int number) {
    assertTrue(calc.isEven(number));
}
```

Supported Types: ints, strings, doubles, longs, booleans, shorts, bytes, chars, floats, classes

3. @MethodSource (Dynamic Data)

When to Use: Complex test data, reusable data sets

Reference: [Calculator Test.java:93](#)

```
// Data provider method
private static Stream<Arguments> provideDivisionTestCases() {
    return Stream.of(
        Arguments.of(20, 4, 5),
        Arguments.of(10, 2, 5)
    );
}

@ParameterizedTest
@MethodSource("provideDivisionTestCases")
void testDivide(int a, int b, int expected) {
    assertEquals(expected, calc.divide(a, b));
}
```

Rules: - Method must be static - Must return Stream<Arguments> - Method name matches annotation value

4. @CsvFileSource (External Data)

When to Use: Large data sets, data managed by QA team

Reference: [Calculator Test.java:113](#)

```
@ParameterizedTest
@CsvFileSource(resources = "/resources/calculator-data.csv", numLinesToSkip
    = 0)
void testAddUsingCsvFile(int a, int b, int expected) {
    assertEquals(expected, calc.add(a, b));
}
```

CSV File Format: [calculator-data.csv](#)

2,3,5

10,5,15
20,4,24

Advantages: - ✓ Separate test data from code - ✓ Easy to add/modify test cases - ✓ Can be version controlled separately

❏ @NullSource, @EmptySource, @NullAndEmptySource

When to Use: Testing null/empty handling

Reference: Calculator Test.java:120

```
@ParameterizedTest
@NullSource
void testNull(Integer value) {
    assertNull(value); // value will be null
}

@ParameterizedTest
@EmptySource
void testEmpty(String value) {
    assertTrue(value.isEmpty()); // value will be ""
}

@ParameterizedTest
@NullAndEmptySource
void testBoth(String value) {
    assertTrue(value == null || value.isEmpty());
}
```

❏ @EnumSource (Enum Testing)

When to Use: Testing all enum values

Reference: Calculator Test.java:149

```
enum Operation { ADD, SUBTRACT, MULTIPLY, DIVIDE }

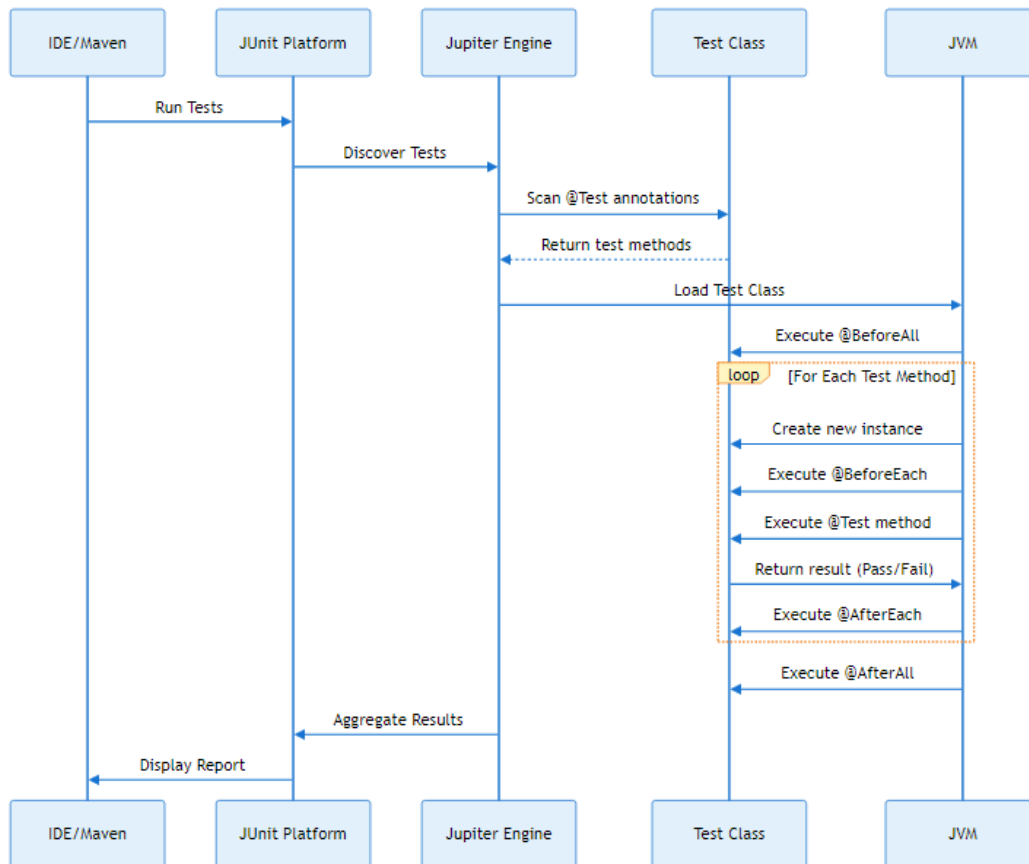
@ParameterizedTest
@EnumSource(Operation.class)
void testEnum(Operation op) {
    assertNotNull(op); // Runs 4 times (one per enum)
}
```

}

7. INTERNAL EXECUTION FLOW

🔍 **Deep Dive by:** Avinash Dhanuka | Understanding JVM Internals

🏠 How JUnit Executes Tests (JVM Level)



🔍 Step-by-Step Execution

Example Test: `Calculator_Test.testAdd()`

1. Class Loading:

JVM loads `Calculator_Test.class` into memory

2. Static Initialization:

```
static int passed = 0; // Initialized once
@BeforeAll static void beforeAllTests() { ... } // Runs once
```

3. Instance Creation (Per Test):

```
Calculator_Test testInstance = new Calculator_Test(); // Fresh instance
```

4. Setup:

```
@BeforeEach void setup() {
    calc = new Calculator(); // New Calculator object
}
```

5. Test Execution:

```
@Test void testAdd() {
    assertEquals(5, calc.add(2, 3)); // Assertion checked
}
```

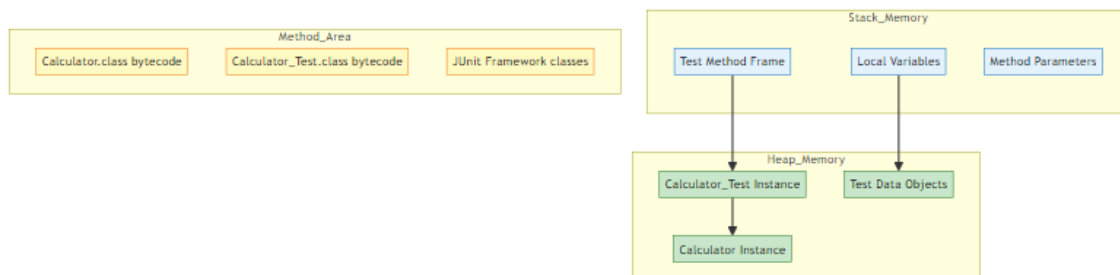
6. Teardown:

```
@AfterEach void tearDown() {
    calc = null; // Cleanup
}
```

7. Final Cleanup:

```
@AfterAll static void afterAllTests() {
    System.out.println("Tests: " + passed); // Summary
}
```

❖ Memory Architecture During Testing



Key Points: - Each `@Test` method gets a **new instance** of the test class - Static variables (passed, failed) persist across all tests - `@BeforeAll` and `@AfterAll` execute in **static context** (no instance)

🔗 TestWatcher Extension (Advanced)

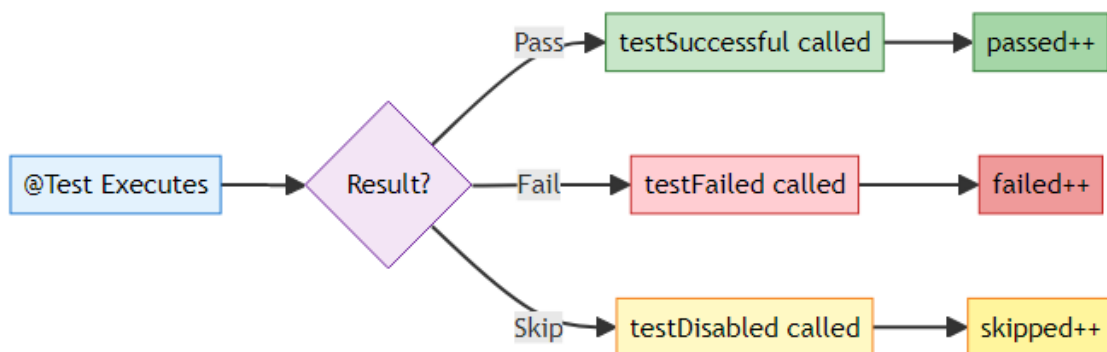
Reference: [Calculator_Test.java:21](#)

```
@RegisterExtension
TestWatcher watcher = new TestWatcher() {
    @Override
    public void testSuccessful(ExtensionContext context) {
        passed++; // Increment on success
    }

    @Override
    public void testFailed(ExtensionContext context, Throwable cause) {
        failed++; // Increment on failure
    }
};
```

What is TestWatcher? - An **Extension** that hooks into test lifecycle - Allows custom logic after each test completes - Used here to track pass/fail/skip counts

Execution Flow:



8. TOPICS COVERED IN THIS PROJECT

✓ Complete Checklist

🔗 Core JUnit Concepts

- ✓ **Test Class Structure:** Proper naming convention (ClassName_Test)
- ✓ **Package Organization:** Mirroring production code structure
- ✓ **Lifecycle Annotations:** @BeforeAll, @BeforeEach, @AfterEach, @AfterAll
- ✓ **Test Annotations:** @Test, @DisplayName, @Disabled, @RepeatedTest

🔍 Assertion Methods (11 Types)

- ✓ assertEquals - Value equality
- ✓ assertNotEquals - Value inequality
- ✓ assertTrue - Boolean true check
- ✓ assertFalse - Boolean false check
- ✓ assertNull - Null check
- ✓ assertNotNull - Not null check
- ✓ assertEquals - Reference equality
- ✓ assertNotSame - Reference inequality
- ✓ assertThrows - Exception testing
- ✓ assertAll - Grouped assertions
- ✓ fail - Manual test failure

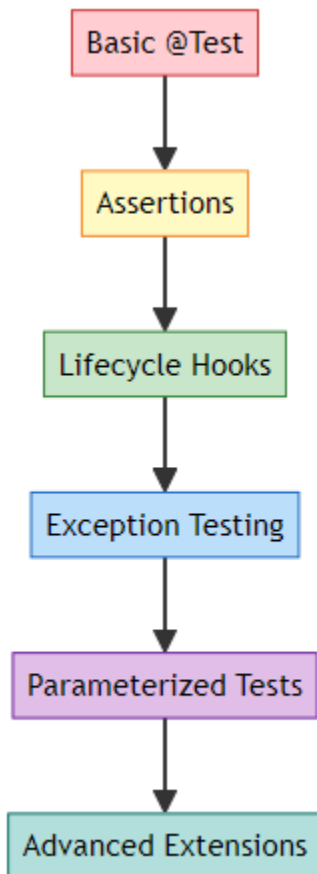
🎲 Parameterized Testing (9 Sources)

- ✓ @CsvSource - Inline CSV data
- ✓ @ValueSource - Single parameter arrays
- ✓ @MethodSource - Dynamic data from methods
- ✓ @CsvFileSource - External CSV files
- ✓ @NullSource - Null value testing
- ✓ @EmptySource - Empty value testing
- ✓ @NullAndEmptySource - Combined null/empty
- ✓ @EnumSource - Enum value testing

🔧 Advanced Features

- ✓ **TestWatcher Extension:** Custom test result tracking
 - ✓ **@RegisterExtension:** Extension registration
 - ✓ **Exception Handling:** Testing expected exceptions
 - ✓ **Test Resources:** External data files
 - ✓ **Static vs Instance Methods:** Understanding context
-

📖 Learning Path Progression



Beginner → Intermediate → Advanced

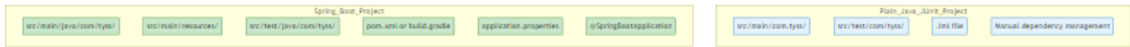
🎓 Real-World Applications

Concept	Real-World Use Case
@CsvSource	Testing payment calculations with different amounts
@MethodSource	Testing user authentication with various credentials
@CsvFileSource	Testing data import from Excel/CSV files
assertThrows	Validating API error responses
@BeforeEach	Setting up database connections before each test
TestWatcher	Generating custom test reports for stakeholders
@RepeatedTest	Stress testing concurrent operations

9. SPRING BOOT VS PLAIN JAVA STRUCTURE

📄 Detailed Comparison by: Avinash Dhanuka | [Connect on GitHub](#)

🏗️ Architectural Comparison



📊 Detailed Comparison Table

Aspect	Plain Java (This Project)	Spring Boot
Project Type	Simple Java Application	Enterprise Web Application
Build Tool	None (IDE managed)	Maven/Gradle (pom.xml/build.gradle)
Dependency Management	Manual JAR files	Automatic via Maven Central
Main Folder	src/main/com.tyss/	src/main/java/com/company/
Resources	src/test/resources/ only	src/main/resources/ +

Aspect	Plain Java (This Project)	Spring Boot
Folder		src/test/resources/
Configuration	None	application.properties Or application.yml
Entry Point	public static void main()	@SpringBootApplication class
Dependency Injection	Manual object creation	@Autowired, @Component
Testing	JUnit 5 (Jupiter)	JUnit 5 + Spring Test (@SpringBootTest)
Web Layer	Not applicable	Controllers (@RestController)
Database	Not applicable	JPA, Hibernate (@Entity, @Repository)
Packaging	.jar (plain)	.jar (executable with embedded Tomcat)

🔍 Folder Structure Comparison

Plain Java (Current Project)

```

JUnitOne/
├── src/
│   ├── main/
│   │   ├── com.tyss/
│   │   │   ├── Calculator.java
│   │   │   └── StudentEligibilityService.java
│   │   └── test/
│   │       ├── com.tyss/
│   │       └── resources/
│   └── out/
└── JUnitOne.iml

```

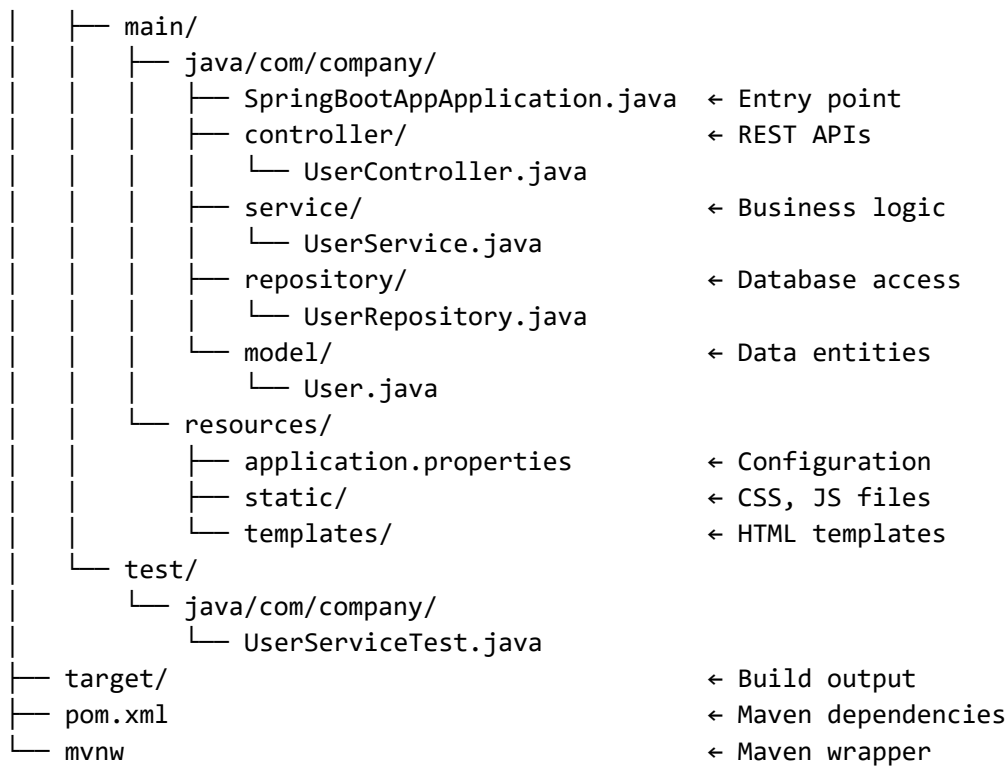
← Business logic only
 ← Test classes
 ← Test data files
 ← Compiled classes
 ← IntelliJ config

Spring Boot Project

```

spring-boot-app/
├── src/

```



🔗 Key Differences Explained

1. Dependency Management

Plain Java:

```
// Manual import of JUnit JAR
import org.junit.jupiter.api.Test;
```

You must download junit-jupiter-5.x.x.jar and add to classpath manually.

Spring Boot (pom.xml):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Maven automatically downloads JUnit + Mockito + AssertJ + Spring Test.

2. Object Creation

Plain Java:

```
@BeforeEach
void setup() {
    calc = new Calculator(); // Manual instantiation
}
```

Spring Boot:

```
@Autowired
private Calculator calc; // Spring creates and injects automatically
```

3. Testing Approach

Plain Java (Unit Test):

```
@Test
void testAdd() {
    Calculator calc = new Calculator();
    assertEquals(5, calc.add(2, 3)); // Pure unit test
}
```

Spring Boot (Integration Test):

```
@SpringBootTest
class UserServiceTest {
    @Autowired
    private UserService userService; // Tests with real Spring context

    @Test
    void testCreateUser() {
        User user = userService.createUser("John");
        assertNotNull(user.getId()); // Tests database interaction
    }
}
```

4. Application Entry Point

Plain Java:

```
public class Calculator {
    public static void main(String[] args) {
        System.out.println("Hello"); // Simple console app
    }
}
```

Spring Boot:

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args); // Starts web
        server
    }
}
```

🌐 When to Use What?

Use Case	Choose
Learning Java basics	Plain Java
Simple utility libraries	Plain Java
Command-line tools	Plain Java
REST APIs	Spring Boot
Microservices	Spring Boot
Database applications	Spring Boot
Enterprise applications	Spring Boot

🔄 Migration Path



Steps to Convert This Project to Spring Boot: 1. Create pom.xml with Spring Boot dependencies 2. Move com.tyss to src/main/java/com/tyss/ 3. Add @Component to Calculator and StudentEligibilityService 4. Create @SpringBootApplication main class 5. Change tests to use @SpringBootTest and @Autowired

10. TOP INTERVIEW QUESTIONS

🐾 **Curated by:** Avinash Dhanuka | © 2026 | [GitHub](#)

🔹 JUnit Fundamentals

Q1: What is the difference between JUnit 4 and JUnit 5?

Feature	JUnit 4	JUnit 5
Architecture	Monolithic	Modular (Platform + Jupiter + Vintage)
Annotations	@Before, @After	@BeforeEach, @AfterEach
Assertions	org.junit.Assert	org.junit.jupiter.api.Assertions
Test Instance	One per class	One per method (default)
Parameterized	@RunWith(Parameterized.class)	@ParameterizedTest
Lambda Support	No	Yes (assertThrows(() -> ...))

Q2: Why does JUnit create a new instance for each test method?

Answer: To ensure **test isolation**. If tests shared the same instance, one test could modify state that affects another test, causing unpredictable failures.

Example:

```
class MyTest {
    int counter = 0; // Instance variable

    @Test void test1() { counter++; } // counter = 1
    @Test void test2() { counter++; } // counter = 1 (new instance!)
}
```

Each test gets counter = 0 because JUnit creates a fresh instance.

Q3: What is the difference between assertEquals and assertEquals?

Answer: - **assertEquals:** Compares **values** using .equals() method - **assertEquals:** Compares **memory references** using == operator

Example:

```
String s1 = new String("hello");
String s2 = new String("hello");

assertEquals(s1, s2);    // ✓ PASS (values are equal)
assertEquals(s1, s2);    // ✗ FAIL (different objects in heap)
```

Q4: When should you use @BeforeAll vs @BeforeEach?

Annotation	Use Case	Example
@BeforeAll	Expensive setup (once)	Database connection, load config file
@BeforeEach	Fresh state (per test)	Create new object instances, reset variables

Reference: [Calculator Test.java:46](#)

Q5: How do you test if a method throws an exception?

Answer: Use assertEquals()

Reference: [Calculator Test.java:208](#)

```
@Test
void testDivideByZero() {
    assertEquals(ArithmeticException.class, () -> {
        calc.divide(10, 0);
    });
}
```

Why Lambda? JUnit needs to execute the code and catch the exception. Lambda delays execution until JUnit is ready.

🔗 Parameterized Testing

Q6: What is the advantage of `@MethodSource` over `@CsvSource`?

Answer:

Feature	@CsvSource	@MethodSource
Data Type	Strings only	Any Java object
Complexity	Simple values	Complex objects, lists
Reusability	No	Yes (method can be reused)
Dynamic Data	No	Yes (can read from DB/API)

Example:

```
// @MethodSource can return complex objects
private static Stream<Arguments> provideUsers() {
    return Stream.of(
        Arguments.of(new User("John", 25)),
        Arguments.of(new User("Jane", 30))
    );
}
```

Q7: How does `@CsvFileSource` find the CSV file?

Answer: It looks in the **classpath** under `src/test/resources/`

Reference: [Calculator Test.java:113](#)

```
@CsvFileSource(resources = "/resources/calculator-data.csv")
```

Path Resolution: 1. JUnit looks in `src/test/resources/resources/calculator-data.csv` 2. During build, files are copied to `out/test/resources/` 3. JVM loads from classpath

Q8: Can you use multiple parameterized sources on one test?

Answer: No, but you can combine some:


```
// ✔ VALID: Combines null and empty
@ParameterizedTest
@NullAndEmptySource
@ValueSource(strings = {" ", "\t", "\n"})
void testBlankStrings(String input) {
    assertTrue(input == null || input.trim().isEmpty());
}
```

🔧 Advanced Concepts

Q9: What is a TestWatcher and when would you use it?

Answer: A **TestWatcher** is a JUnit extension that hooks into test lifecycle events.

Use Cases: - Custom logging - Screenshot capture on test failure (Selenium) - Performance metrics tracking - Custom reporting

Reference: [Calculator_Test.java:21](#)

Q10: Why are @BeforeAll and @AfterAll methods static?

Answer: Because they run **before any instance is created**. JUnit needs to call them without instantiating the test class.

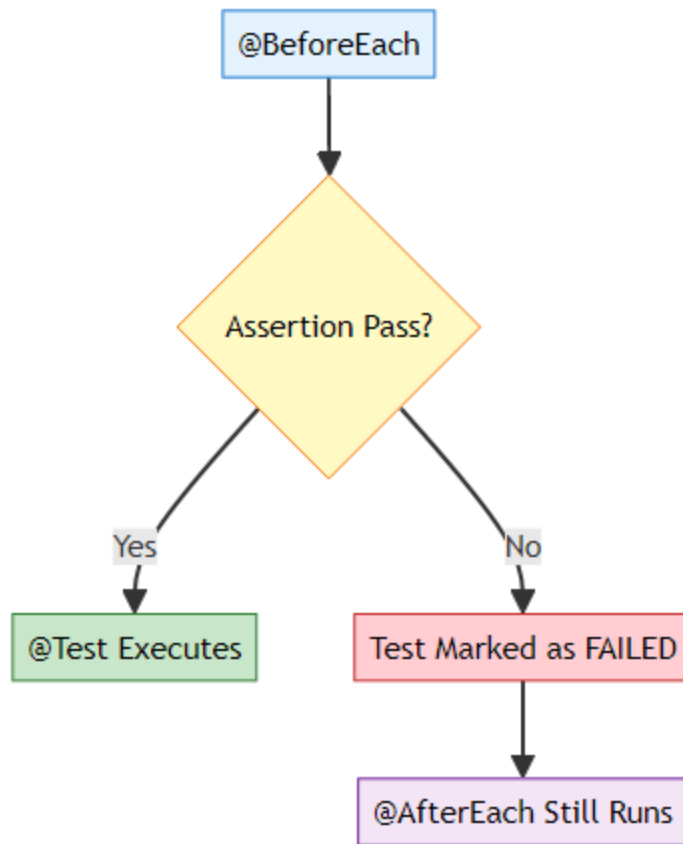
Memory Timeline:

1. Load Calculator_Test.class into Method Area
 2. Execute @BeforeAll (static context)
 3. Create instance #1 → run test1
 4. Create instance #2 → run test2
 5. Execute @AfterAll (static context)
-

Q11: What happens if an assertion fails in @BeforeEach?

Answer: The test is **skipped** and marked as **failed**. The @Test method never executes.

Execution Flow:



Q12: How do you skip a test conditionally?

Answer: Use `Assumptions.assertTrue()`

```
@Test
void testOnlyOnWindows() {
    Assumptions.assertTrue(System.getProperty("os.name").contains("Windows"));

    // Test only runs on Windows
}
```

Difference from @Disabled: - @Disabled: Always skips - Assumptions: Skips based on runtime condition

📖 Scenario-Based Questions

Q13: You have 100 test methods. How do you run only tests related to “addition”?

Answer: Use @Tag annotation

```
@Test
@Tag("addition")
void testAdd() { ... }

@Test
@Tag("division")
void testDivide() { ... }
```

Run Command:

```
mvn test -Dgroups="addition"
```

Q14: How would you test a method that takes 10 seconds to execute?

Answer: Use @Timeout annotation

```
@Test
@Timeout(value = 5, unit = TimeUnit.SECONDS)
void testSlowMethod() {
    slowService.process(); // Fails if takes > 5 seconds
}
```

Q15: Your test passes locally but fails in CI/CD. What could be wrong?

Possible Causes: 1. **Environment Variables:** Missing in CI 2. **File Paths:** Absolute paths instead of relative 3. **Time Zones:** Date/time assertions 4. **Test Order Dependency:** Tests not isolated 5. **Resource Files:** Not included in build

Solution: Use @TestInstance(Lifecycle.PER_CLASS) to debug state issues.

💡 Best Practices

Q16: What are the characteristics of a good unit test?

FIRST Principles: - **Fast:** Runs in milliseconds - **Independent:** No dependencies between tests - **Repeatable:** Same result every time - **Self-Validating:** Pass/fail, no manual verification - **Timely:** Written before or with production code (TDD)

Q17: Should you test private methods?

Answer: No. Test public methods only. Private methods are implementation details.

Reasoning: - Private methods are tested indirectly through public methods - Testing private methods couples tests to implementation - Refactoring becomes difficult

If you must test private methods:

```
// Use reflection (not recommended)
Method method = Calculator.class.getDeclaredMethod("privateMethod");
method.setAccessible(true);
method.invoke(calc);
```

🔪 Real-World Scenarios

Q18: How do you test a REST API controller in Spring Boot?

Answer: Use @WebMvcTest + MockMvc

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    private MockMvc mockMvc;

    @Test
    void testGetUser() throws Exception {
        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("John"));
    }
}
```

```
}  
}
```

Q19: How do you test database operations?

Answer: Use `@DataJpaTest` with H2 in-memory database

```
@DataJpaTest  
class UserRepositoryTest {  
    @Autowired  
    private UserRepository repository;  
  
    @Test  
    void testSaveUser() {  
        User user = new User("John");  
        User saved = repository.save(user);  
        assertNotNull(saved.getId());  
    }  
}
```

Q20: What is the difference between Unit Test and Integration Test?

Aspect	Unit Test	Integration Test
Scope	Single method/class	Multiple components
Dependencies	Mocked	Real (database, APIs)
Speed	Fast (milliseconds)	Slow (seconds)
Annotation	@Test	@SpringBootTest
Example	Testing Calculator.add()	Testing REST API → Service → Data- base

This Project: Pure **Unit Tests** (no external dependencies)

End of Master Guide

Created with dedication by Avinash Dhanuka

© 2026 Avinash Dhanuka | All Rights Reserved

This documentation is protected intellectual property. Unauthorized reproduction or distribution is prohibited.

Happy Testing! 🕒

"Write tests, not bugs!" - Avinash Dhanuka